

1 Background

This section attempts to go through the basic concepts of MDP and POMDP that might be useful for this lab.

1.1 Markov Decision Process

MDP is a discrete-time control system whose transitions are probabilistic. In MDP, the states are fully observable, and only the control effects are uncertain. We can represent this uncertainty through state transition function. In MDP, we will have states $\mathcal{S}$, action $\mathcal{A}$, transition probability $P(s'|s, a)$ and reward function $r(s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$.

A policy $\pi : \mathcal{S} \times \mathcal{A}$ is a mapping from states to actions, determining what action to take from each possible state. Our goal in solving MDP is to either maximizing the cumulative reward, or minimizing the cost, and thus the optimal policy can be written as:

$$\begin{aligned}\pi^* &= \arg \max_{\pi} J(s_0, (\pi(s_0), \pi(s_1), \dots)) \\ &= \mathbb{E}_{s_0, s_1, \dots} \sum_{t_0}^{\infty} \gamma^t r(s_t, \pi(s_t))\end{aligned}$$

In MDP, value function is a function of state, $V(s)$ that defines the expected value of a state given a defined policy. And if we apply the optimal policy π^* , we will receive the maximum value of a state, denoted $V^*(s)$. The value function can be written as:

$$V(s_0) := \max_{\pi} \mathbb{E}_{s_1, s_2, \dots} \sum_{t_0}^{\infty} \gamma^t r(s_t, \pi(s_t))$$

From this, we can rewrite the equation for value function as:

$$V(s_0) = \max_{a_0} \mathbb{E}_{s_1} [r(s_0, a_0) + \gamma V(s_1)] \quad (1)$$

Equation (1) is the Bellman Equation for MDP that can be solved using Dynamic Programming.

1.1.1 Value iteration

Value iteration is a process in which we apply the Bellman Equation to iteratively update the value of all states. We modify (1) into:

$$V^{k+1} = \max_{\pi} [r_{\pi} + \gamma P_{\pi} V^k] \quad (2)$$

Using (2), we have the following pseudo-code for Value Iteration:

Algorithm 1 Value Iteration

Require: $P(\cdot), r(\cdot)$ (state transition function/reward function), γ, ϵ .
1: $V^0 \leftarrow \mathbb{R}^N$ ▷ Start with any guess
2: **while** not $\|V^{k+1} - V^k\|_\infty < \epsilon$ **do**
3: **for** s in $\mathcal{S}$ **do** ▷ Update new value for all states
4: $V^{k+1}(s) \leftarrow \max_\pi [r(s, \pi(s)) + \gamma P_\pi V^k(s)]$
5: **end for**
6: **end while**
7: **return** V

Once we got the converged value from *Value Iteration*, the optimal policy π^* can be computed as:

$$a_s^* := \pi^*(s) = \arg \max_a r(s, a) + \gamma P_a V \quad (3)$$

Equation (3) says that the optimal action to take at each state is the action that maximizes the immediate reward and discounted cumulative reward, among all the actions that can be taken from the respective state.

1.1.2 Policy iteration

In policy iteration, you start off with a candidate/guess policy π . Then, using that policy, we will calculate the value for all states, and use the state values to derive a new policy. The iterative process then follows this pseudo-code:

Algorithm 2 Policy Evaluation

Require: π (current policy), $P(\cdot), r(\cdot), \gamma, \epsilon$.
1: **while** not $\|V^{k+1} - V^k\|_\infty < \epsilon$ **do**
2: **for** s in $\mathcal{S}$ **do**
3: $V^{k+1}(s) \leftarrow \sum P_\pi [r(s, \pi(s)) + \gamma V^k(s)]$
4: **end for**
5: **end while**
6: **return** V

Algorithm 3 Policy Iteration

Require: $P(\cdot), r(\cdot)$ (state transition function/reward function), γ, ϵ .

- 1: $\pi(s) \leftarrow \mathbb{R}^N$ ▷ Start with any candidate/guess policy
- 2: $policy_stable \leftarrow True$
- 3: $V \leftarrow policy_evaluation(\cdot)$ ▷ Get the current state value with initialized policy
- 4: **while** $True$ **do**
- 5: **for** s in $\mathcal{S}$ **do**
- 6: $old_action \leftarrow \pi(s)$
- 7: $\pi(s) \leftarrow \arg \max_a \sum P_\pi[r(s, \pi(s)) + \gamma V(s)]$
- 8: **if** $old_action \neq \pi(s)$ **then**
- 9: $policy_stable \leftarrow False$
- 10: **end if**
- 11: **end for**
- 12: **if** not $policy_stable$ **then**
- 13: $V \leftarrow policy_evaluation(\cdot)$ ▷ Calculate new state value using new policy
- 14: $policy_stable \leftarrow True$
- 15: **else**
- 16: **break**
- 17: **end if**
- 18: **end while**
- 19: **return** V, π

Policy iteration and value iteration, if implemented correctly, will result in similar optimal policy.

1.2 Partially Observable Markov Decision Process

In POMDP, we can apply similar idea in MDP. However, the state s is not observable. The robot has to make its decision in the belief state, which is the space of posterior distributions over states.

POMDP computes a value function over belief space:

$$V_T(b) = \max_u [r(b, a) + \int \gamma V_{T-1}(b') p(b'|a, b) db'] \quad (4)$$

With $V_0(b) = \gamma \max_a \mathbb{E}_x[r(s, a)]$. The induced control policy is as follows:

$$\pi_T(b) = \arg \max_a [r(b, a) + \int \gamma V_{T-1}(b') p(b'|a, b) db'] \quad (5)$$

A belief is a probability distribution; thus, each value in a POMDP is a function of an entire probability distribution. This is problematic. If the state space is finite, the belief space is continuous, since it is the space of all distributions over the state space. Thus, there is a continuum of different values, whereas there was only a finite number of different values in the MDP case. The situation is even more intractable when we are dealing with continuous state spaces, as the belief space will be an infinitely-dimensional continuum.

However, when the world we are considering is finite, that is, the state space, the action space, the space observations and the planning horizon are all finite, an exact solution exists. Though solution can be calculated in this case, the computational expense of exact planning makes it inapplicable to use with almost any practical problems in robotics.

1.3 QMDP

Instead of computing the exact solution, we can approximate it. One of the methods to do so is *QMDP*. *QMDP* is a hybrid between MDP and POMDP. This algorithm generalizes the MDP-optimal value function defined over states, into a POMDP-style value function over beliefs. QMDP works under the assumption that after one step of control, the state becomes fully observable.

The following pseudo-code shows how QMDP works:

Algorithm 4 QMDP

Require: $b = (p_1, p_2, \dots, p_N)$ (beliefs), $P(\cdot), r(\cdot)$ (state transition function/reward function of MDP), γ, ϵ .

- 1: $\hat{V} \leftarrow \text{MDP-value-iteration}(\cdot)$
- 2: **for** a in $\mathcal{A}$ **do**
- 3: $Q(s_i, a) \leftarrow r(s_i, a) + \sum_{j=1}^N \hat{V}(s_j) P(s_j|a, s_i)$
- 4: **end for**
- 5: **return** $\arg \max_a \sum_{i=1}^N p_i Q(s_i, a)$

The mathematical trick of *QMDP* is relatively straightforward. The MDP provides us with a state-based value function that is optimal under the assumption that state is fully observable. The resulting value function $\hat{V}$ is defined over world states. The *QMDP* generalizes this value to the belief space through the mathematical expectation:

$$\hat{V}(b) = \mathbb{E}_s[\hat{V}(s)] = \sum_{i=1}^N p_i \hat{V}(s_i) \quad (6)$$

With $p_i = b(s_i)$. It can also be observed from Algorithm 4 that the computation on line 3 of the algorithm is computing the MDP-value of taking an action a at a certain state s_i . We then generalize the value to the belief states, by taking the expectation over the belief state. We finally maximize over all actions, and return the control action with the highest expected value.

1.3.1 Bayesian Inference

Bayesian Inference can be used to update the belief space in QMDP when new observations are received. Let's revise this a little. Combining the definition of conditional probability with the product and sum rules yields **Bayes rule**, also called **Bayes Theorem**:

$$p(X = x|Y = y) = \frac{p(X = x, Y = y)}{p(Y = y)} = \frac{p(X = x)p(Y = y|X = x)}{\sum_{x'} p(X = x')p(Y = y|X = x')} \quad (7)$$

Let's look at an example of this. Consider the following medical diagnosis problem: Suppose a person is testing for cancer. This person is told that the test has a **sensitivity** of 80%. This means that if this person has cancer, the test will be positive with probability 0.8 (true positive). In other words,

$$p(x = 1|y = 1) = 0.8 \quad (8)$$

where $x = 1$ is the event that test is positive, and $y = 1$ is the event this person actually has cancer. Many people will conclude that this person has 80% chance to have cancer. But this is not the case. Doing so will ignore the prior probability of having this type of cancer, which, assuming and fortunately, is low:

$$p(y = 1) = 0.004 \quad (9)$$

Ignoring this prior is called the **base rate fallacy**. We also need to take in account that the test may be a **false positive** or **false alarm**. Unfortunately, such false positives are quite likely (with current technology):

$$p(x = 1|y = 0) = 0.1 \quad (10)$$

Combining these three terms using Bayes rule, we can compute the correct answer as follows:

$$\begin{aligned} p(y = 1|x = 1) &= \frac{p(x = 1|y = 1)p(y = 1)}{p(x = 1|y = 1)p(y = 1) + p(x = 1|y = 0)p(y = 0)} \\ &= \frac{0.8 \times 0.004}{0.8 \times 0.004 + 0.1 \times 0.996} = 0.031 \end{aligned}$$

In other words, if this person test comes out as positive, it would mean that there is only 3% chance of actually having cancer (which is now less scary in comparison to the original 80%).

1.3.2 Dirichlet-multinomial model

The Dirichlet-multinomial model uses Bayes Theorem to gradually update our belief for a multinomial distribution. Using Dirichlet distribution as prior for multinomial distribution in Bayesian Inference is helpful in computation, as Dirichlet is a **conjugate prior** of multinomial, meaning that the calculated posterior distribution is in the same probability distribution family as the prior probability distribution (Dirichlet, in this case).

Consider a dice with K sides, we want to infer the probability of this dice giving is the face k . Suppose we observe N dice rolls, $\mathcal{D} = \{x_1, \dots, x_N\}$ where $x_i \in \{1, \dots, K\}$. If we assume that the data is iid, the likelihood has the form:

$$p(\mathcal{D}|\theta) = \prod_{k=1}^K \theta_k^{N_k}$$

Where N_k is the number of times event k occurred.

We will use Dirichlet distribution as our conjugate prior. Define the Dirichlet prior as:

$$\text{Dir}(\theta|\alpha) = \frac{1}{B(\alpha)} \prod_{k=1}^K \theta_k^{\alpha_k-1}$$

Multiplying the likelihood by the prior, we find that the posterior is also Dirichlet:

$$\begin{aligned} p(\theta|\mathcal{D}) &\propto p(\mathcal{D}|\theta)p(\theta) \\ &\propto \prod_{k=1}^K \theta_k^{N_k} \theta_k^{\alpha_k-1} = \prod_{k=1}^K \theta_k^{\alpha_k+N_k-1} \\ &\propto \text{Dir}(\theta|\alpha_1 + N_1, \dots, \alpha_K + N_K) \end{aligned}$$

We can see that the posterior is obtained by adding the prior hyper-parameters (pseudo-counts) α_k to the empirical counts N_k .

From here, we can derive the posterior predictive distribution for a single multinoulli trial, given by the following expression:

$$p(X = j|\mathcal{D}) = \frac{\alpha_j + N_j}{\alpha_0 + N}$$

With $\alpha_0 \equiv \sum_{k=1}^K \alpha_k$, the sample size of the prior.

Let's walk through an example to understand this easier. Assuming that we want to know the probability of each face of a 6-face dice. We will model the distribution with Dirichlet-multinomial. Let us have an unbiased prior belief on the distribution of the faces, that is, every face has the same initial probability of $\theta_j = \frac{1}{6}$ ($\alpha = \{1, 1, 1, 1, 1, 1\}$).

We start throwing the dice 10 times, and record the values, $\mathcal{D} = \{1, 2, 1, 1, 2, 5, 4, 2, 1, 2\}$. Using the above posterior predictive equation, we now have the following updated probability for each face:

$$\begin{aligned} p(X = j|\mathcal{D}) &= \frac{\alpha_j + N_j}{\sum_i \alpha_i + N} = \frac{1 + N_j}{6 + 10} \\ &= \left(\frac{5}{16}, \frac{5}{16}, \frac{1}{16}, \frac{2}{16}, \frac{2}{16}, \frac{1}{16} \right) \end{aligned}$$